# An Accurate Arabic Lemma-to-Root Extraction

Technical report on work done in 2012–2013, re-run and revised in 2026

Omar Duhaiby

**Abstract**

In this report, a model is proposed for finding the trilateral (three-lettered) Arabic root of any given noun lemma. The model is trained using the LibSVM library. We present and discuss previous work. Then we explain two different approaches: finding the whole root as a string, and finding it letter by letter; the latter showed significantly better results and has reduced dimensionality than the former. We also discuss the data preprocessing, the feature sets, a layered variant in which each classifier passes its answer to the next, and the experimental results, all of which were re-run in 2026 under one protocol.

## 1 Introduction

Arabic is spoken by about 350 million people worldwide. In terms of internet usage, it scored the highest growth rate, 2,501.2% for the period 2000–2010 (Brahmi et al., 2011). In addition, other languages such as Farsi, Malay and Urdu write using the Arabic script (Al-Serhan and Ayesh, 2006b). And even though Arabic natural language processing (ANLP) research still has not reached serious maturity as have other Latin-based languages, interest in it has grown much quicker than in other fields of computational linguistics; and it will grow even quicker with all its resources emerging and being cleaned up, honed and represented.

With the frighteningly growing number of Arabic text on the internet, most research interest is going towards the field of semantic analysis. Subjects like Sentiment Analysis, Information Retrieval (IR), Machine Translation (MT), Summarization and Keyphrase Extraction, are gaining more popularity amongst researchers, even though basic tools required for such high-level tasks have not yet matured. Such basic tools include Text Diacritization (restoring vowels), Morphological Analysis, and Root Extraction. These tools, although having reached a stage of saturation in terms of research, are still in their infancy stages in Arabic in terms of implementation. The main reason is the difficulty posed by the highly inflectional properties of Semitic languages, and especially Arabic (Brahmi et al., 2011; Altantawy et al., 2011; Al-Serhan and Ayesh, 2006b).

Unlike Latin-based languages that share enough similarities with English to allow semantic-analysis algorithms written for English to work with them very efficiently, Arabic, because of its high inflectional morphology, makes such attempts fail so miserably that the task seems virtually impossible. This is evident in the most advanced Machine Translation and Information Extraction implementations, e.g. Google's; it uses the same methods on Arabic as those on English. A more effective and appealing solution is to work with such texts in their original language (Brahmi et al., 2011).

Root extraction is one of the problems at the heart of the language. Every Arabic word that is not a particle or a loanword is built on a root, and finding that root is the first thing a reader of Arabic does with an unfamiliar word. That is reason enough to want a machine that does it well, whether or not a particular application is waiting for it.

This report is outlined as follows. Section 2 explains necessary background information. Section 3 reviews recent related works in the literature. Sections 4 and 5 explain the design and technical details of the implementation. Section 6 shows the results. Section 7 describes the first application of the extractor. We discuss future work in Section 8 and conclude with Section 9.

## 2 Background

Semitic languages are morphologically rich, i.e., by applying derivation patterns alone, many derivatives can be produced. A *root* is a sequence of consonants, three by default, called its *radicals*. A *pattern* is a sequence of vowels, one for each of the root's radicals, and a limited set of letters with their vowels. The pattern is plugged onto the root, producing a *lemma* (Daya et al., 2004). So a lemma consists of two morphemes: a root and a pattern. Arabic has more than 10,000 roots (Daya et al., 2004), 85% of which are three-lettered (Al Hajjar et al., 2010). A maximum of 30 derivatives can be produced from one root using different patterns. This makes Arabic even richer morphologically than other Semitic languages. Morphological inflections can further be applied on the lemmas, adding prefixes or suffixes, called *inflectional morphemes*, to produce even more diverse forms we call stems. Broken plural patterns can be applied as well on lemmas to produce more stems.

There is a thin line between lemmas and stems. A lemma can act as a stem without adding inflectional morphemes or applying broken plural patterns, but it can also make different stems by performing exactly these two processes. To simplify it, every lemma is a stem, but not every stem is a lemma.

Now stems, including lemmas that have not been inflected, can still be further modified by adding clitics, again as prefixes or suffixes, but that is just it; this is the final stage of the morphological processes. It produces the multi-morpheme tokens we encounter in regular Arabic text. Again, it is not necessary for clitics to be added to either a lemma or a stem for it to make sense. Figure 1 sums up this whole journey.

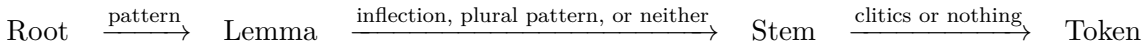



Figure 1: Stages of Arabic morphology.

Aided by Figure 1, dealing with the definitions of morphological tasks will be straightforward.

- **Stemming** is the process by which a stem is extracted from a token by removing the clitical morphemes. Stemming is not the same as root extraction. In fact, it is a prerequisite of root extraction.
- **Lemmatization** is simply extracting the lemma from the stem or, in some cases, directly from the token.
- **Root extraction** is the process where the three-lettered root is restored from the lemma or the broken-plural stem. This task is tricky; it is not associated with prefixes, suffixes or even infixes. Little research has been done on this problem.

In linguistics, a lemma is the word form that makes dictionary entries. However, that is not the case in Arabic. In an Arabic dictionary or thesaurus, words (lemmas) that share the same root are found under a single entry, which is the root itself. This leads to confusion in English literature caused by the presumption that these two terms, lemma and root, are the same, leading to the belief that root extraction is the same as lemmatization. Not only that, but some authors confuse lemmatization with stemming, because for English, the thin line between the two disappears; and since, in English, root and stem are the same, the confusion automatically leads to the next level, making root = lemma = stem!

Because of such confusion apparent in literature, it takes extra effort to find, filter out, and then analyze the relevant solutions. Also, there is a lack of good journal papers covering the specific subject of root extraction. Not only that, but most of the recent sources that claim to use root extraction actually use stemming or lemmatization, which makes them useless for this survey. The reason is that these sources focus on IR applications; and in IR, lemmatization is easier and more effective than root extraction (Daya et al., 2008).

In this report, we use the terms according to the definitions above, not as they are in cited sources.

### 2.1 Applications

IR applications are the most popular in ANLP. For these, light stemmers are convenient enough (El-Shishtawy and El-Ghannam, 2012). In fact, root extraction would simplify the tokens so much that they would lose their specialized meaning. The same can be said regarding the problem of Text Categorization.

However, our application is concerned directly with the Arabic word's root. A traditional Arabic dictionary uses the root in two ways. First, the root is the index of the dictionary. Every entry is headed by a root. Second, each entry is a nested structure. The entry lists the derivatives of its root. Each derivative carries its meanings, examples and quotations. Converting such a dictionary into a structured lexical database requires rebuilding the index and the nested structure. Rebuilding them requires the root of every word in the dictionary. Looking up a word also requires its root. A reader who meets an unfamiliar word cannot find its entry without first finding its root. Such lexical databases can later help build a general-domain ontology and infer the basic relationships in it.

Roots are also needed by the morphological analyzers that model Arabic's root-and-pattern morphology directly: MAGEAD, for example, defines every lexeme as a root together with a morphological behavior class (Altantawy et al., 2011).

> "Identifying the root of a given word is an important task. Although existing morphological analyzers for Hebrew only provide a lexeme (which is a combination of a root and a pattern), for other Semitic languages, notably Arabic, the root is an essential part of any morphological analysis simply because traditional dictionaries are organized by root, rather than by lexeme. Furthermore, roots are known to carry some meaning, albeit vague." (Daya et al., 2004)

## 3 Related work

The purpose of this survey is to list, explain, and analyze available algorithms in order to build a solid ground of research as a basis for a competitive implementation of a purely machine-learning algorithm that can accurately extract the root from any Arabic word.

We can categorize Arabic root finding algorithms, in general, into machine learning (ML) algorithms and non-ML algorithms.

### 3.1 Non-ML algorithms

A very good survey by Al-Shawakfa et al. (2010) tests six non-ML algorithms. Of the six, four are rule-based, one is a hybrid between database and rule-based, and one is weight-based. We were interested in the weight-based algorithm by Al-Serhan et al. (2003) even though it scored lowest according to the results of Al-Shawakfa et al. (2010). We believe it scored low because it was fed inflected and cliticized words, the type found in regular text, instead of lemmas. Like our work, it was designed to convert lemma to root, without stemming or lemmatization. So it is unfair to feed it more than what it is supposed to handle.

The algorithm by Al-Serhan et al. (2003) is the only weight-based algorithm that Al-Shawakfa et al. (2010) found. It does not use rules, morphological patterns, or a database. Instead of using patterns to know (or estimate, in some ambiguous cases) which letters are radicals and which are not, it uses prefixed weights, assigned to letters, that reflect the behavior of the letter when it is not a radical, and the frequency of this letter occurring as an extra letter. The letters of the input word are the nodes, and the weights are the initial scores.

Even though rules are not explicitly used, they are naturally embedded in the weights. For example, a well-known rule states that there is a closed set of letters that can be added, following a pattern, to a root to make a lemma. To memorize it, traditional scholars of Arabic grouped them into an Arabic word, سألتمونيها, meaning literally "You (plural) asked me for it (feminine)". They are 10 letters, out of 28 of the total alphabet; the rest of the letters are given zero weights.

Other than the weights, the letters are assigned what is called letter-order values. These depend on the position of the letter in the word and the length of the word.

After assigning a letter's letter-order value, the algorithm multiplies it with its weight, finalizing its score, and then scans the word from beginning to end, selecting the lowest scoring letters, and stops when the output reaches three letters.

This algorithm is interesting because it shows effectiveness and accuracy without using a database or rules. Moreover, the idea of weights assigned to letters can be expanded to multidimensional weights; the extra dimension can be the position of the letter in the word, its vowel if it had any, and its neighboring letters. After that, these dimensions can be fed as features to a machine learning algorithm. The letters would have their initial weights as set by Al-Serhan et al. (2003), whereas the training would tweak these weights. This algorithm was the inspiration that led to our feature set design.

### 3.2 ML algorithms

Khorsi (2010) uses a completely unsupervised approach.

Daya et al. (2004) use SNoW, a multi-class classifier that has been successfully used in a number of NLP-related tasks. However, the same authors never mention SNoW in later papers on the same subject (Daya et al., 2008). The features used are both grammatical and statistical:

- Location of letters. Word length is limited to 20, thus obtaining 440 features of this type (recall that the size of the Hebrew alphabet is 22).
- Bigrams of letters (two-letter sequences), independently of their location in the word. This yields 484 features.
- Prefixes. There are 292 features of this type, corresponding to 17 prefixes and sequences thereof.
- Suffixes (e.g., "im", a plural suffix). There are 26 such features.

The first and second features are useful for Arabic. You can see from the fourth feature how easy it is in Hebrew to include plural stems, since there are no broken plurals like in Arabic.

Al-Serhan and Ayesh (2006b) used a Back Propagation Neural Network (BPNN):

> "The input layer consists of 15 nodes (5 letters and each letter represented by 3 bits). One hidden layer consists of 15 nodes, which are fully connected to the input layer, output layer. The output layer consists also of 15 nodes representing the root output."

This is a straightforward design. The same exact model was used in Al-Serhan and Ayesh (2006a) as well.

## 4 Data

The data comes from ALECSO's Sarf database (ALECSO, 2007) and from KACST, the sources the Alkhalil morphological analyzer is built on (Alkhalil). They map roots to their derivatives. For simplicity and ensuring an effective solution, the project was limited to dealing with three-lettered roots only.

### 4.1 Data preprocessing

Preprocessing the data was done manually at first. Then it was integrated in the Java implementation. Basically it was a sequence of find-and-replace operations in order to:

- organize the data into root-derivative pairs;
- remove extra useless beautification characters;
- remove four-lettered roots and their noun derivatives;
- fix some illegal sequences of characters (consecutive consonants);
- replace misplaced diacritics in some special cases;
- convert shadda in roots to a double letter;
- convert the special character آ to its simpler equivalent ءا;
- revert all hamza characters (ء, ئ, ؤ, أ, إ) to one normal form (ء).

### 4.2 Used data analysis

After preprocessing, we got 25,965 derivatives stemming from 5,885 roots. All of these derivatives are of the direct-noun type, i.e., they follow certain patterns and apply only to three-lettered roots. The 30 patterns are listed in Table 1.

| | | | | | |
|---|---|---|---|---|---|
| مَفْعَل | mafʕal | مَفْعِل | mafʕil | فِعالة | fiʕālah |
| فُعال | fuʕāl | فَعيل | faʕīl | فَعْلة | faʕlah |
| فَعْل | faʕl | فَعَل | faʕal | فُعول | fuʕūl |
| تَفْعِيل | tafʕīl | تَفْعِلة | tafʕilah | إفعال | ifʕāl |
| افْتِعال | iftiʕāl | انْفِعال | infiʕāl | تَفَعُّل | tafaʕʕul |
| افْعِلَّال | ifʕillāl | افْعِنْلال | ifʕinlāl | اسْتِفْعَال | istifʕāl |
| فَعَلان | faʕalān | فِعال | fiʕāl | فِعْلة | fiʕlah |
| فُعْلة | fuʕlah | فَعالة | faʕālah | فُعولة | fuʕūlah |
| مفاعلة | mufāʕalah | تَفَعْلُل | tafaʕlul | تَفَاعُل | tafāʕul |
| افْعِلَال | ifʕilāl | افعيعال | ifʕīʕāl | افعِوَال | ifʕiwāl |

Table 1: All 30 patterns of derivatives in our data.

Four-lettered roots were ignored because they represent only 15% of the roots, they are rarely used, and they spawn only two direct-noun derivatives: فَعْلَلة and فِعْلال.

## 5 Features

The straightforward approach is to make each whole root a class. We started with this approach but soon after, we divided the problem into three sub-problems, each concerning finding one radical of the root; and with the division comes dimensionality reduction. The following sections explain.

### 5.1 Problem analysis and dimensionality reduction

The patterns are a closed set which derivatives follow. A derivative consists of radicals, extra letters and short vowels; long vowels are considered extra letters. There are only three radicals in a derivative, which are the exact same letters that constitute the root of that derivative. These radicals are our target letters. Therefore, it would be useful to distinguish which letters come as extra letters in all of the 30 patterns.

The patterns were aligned by the positions of their letters, giving Table 2. No Arabic word conforming to Arabic morphology should be longer than 7 letters. The table shows which extra letters occur at which position, and at which positions each radical (written ف, ع, ل in the patterns) can occur.

| Position | Extra letters | Radicals that can occur here |
|---|---|---|
| 1 | إ ا ت م | first |
| 2 | س ن | first, second |
| 3 | ت ا و ي | first, second, third |
| 4 | ا ن و ي ة | first, second, third |
| 5 | ا ن ة | second, third |
| 6 | ا ة | third |
| 7 | (none) | third |

Table 2: Positions in the word: the extra letters that occur there, and the radicals that can. The first radical occurs only in positions 1–4, the second in 2–5, the third in 3–7.

### 5.2 Used feature sets

For a simple and effective set of features, we chose to make three sets, each for training a classifier on extracting one radical. Table 3 lists the features of all three extractors.

| Radical 1 extractor | Radical 2 extractor | Radical 3 extractor |
|---|---|---|
| length | length | length |
| isLetter$n$InCorrespondingGroup, $n = 1..4$ | …, $n = 2..5$ | …, $n = 3..7$ |
| isLetter$n$Vowel, $n = 1..4$ | …, $n = 2..5$ | …, $n = 3..7$ |
| isLetter$n$Hamza, $n = 1..4$ | …, $n = 2..5$ | …, $n = 3..7$ |
| WhatLetter$n$Haraka, $n = 1..4$ | …, $n = 2..5$ | …, $n = 3..7$ |
| | rootL1Position | rootL1Position, rootL2Position |

Table 3: Feature sets for the three classifiers. isLetter$n$InCorrespondingGroup tells whether letter $n$ belongs to the extra letters of its position (Table 2); WhatLetter$n$Haraka is the diacritic on letter $n$; the last row is used only by the layered classifiers.

The features were extracted from the data using Java. The diacritic features (WhatLetter$n$Haraka) were added later, and gave better results, as shown in Section 6.

### 5.3 Class attribute

The straightforward class set comprises the letters themselves. But since the radicals (the result) are all available in the derived word (the input), we can simply make the class the index of that letter in the input. This reduces the number of classes, for the three separate classifiers, from $28 \times 3 = 84$ to $8 \times 3 = 24$, 28 being the number of letters in the alphabet and 8 the 7 valid positions in the input string plus position 0, described below. For the one-classifier approach,

applying this idea skims down the total number of classes from around 5,000, the total number of roots, to $7 \times 6 \times 5 = 210$.

Each classifier has one more class, position 0. Position 0 means that the radical is not in the word. Each radical can occur only in the positions that Table 2 allows, because no pattern places it anywhere else. A radical that does not occur in its allowed positions therefore has no letter of its own in the word. For example, a weak radical may be written as ا in the derivative. The classifiers detect this case by predicting position 0. Restoring the radical is a separate step with its own rules. This work did not build that step. It is left for future work.

### 5.4 Layered classifiers

The three classifiers can work independently, or in layers: the second classifier is also given the position the first classifier found, and the third is given the positions the first two found. In training, these inputs hold the correct positions from the data. In use, they hold the positions the earlier classifiers predicted; the evaluation also reports an oracle variant that is given the correct positions (Section 6).

## 6 Results

All results in this report come from one re-run of the original 2013 program, carried out in 2026. The features are produced by the 2013 feature extractor, unchanged, from the data file; words longer than 7 letters, which the features cannot represent, are left out, leaving 25,952 words. Each configuration is scored with 10-fold cross-validation, and the three classifiers are tested on the same words in each fold, so that a whole root counts as right only when all three radicals are right for the same word. The classifier is LibSVM (Chang and Lin, 2011) through WEKA, with the 2013 parameters (C-SVC, RBF kernel, $C = 1$).

The configurations vary in three ways. The split is either by word, where words are shuffled into folds, or by root, where all words of a root fall in the same fold, so that the classifiers are always tested on roots they never saw. The classifiers are independent, layered as described above ("chained"), or layered with the correct earlier positions given at test time ("oracle"), which shows how much the errors of the first classifiers cost the later ones. Vocalization is either on, with the diacritic features, or off, without them.

| Split | Classifiers | Vocalization | Radical 1 | Radical 2 | Radical 3 | Whole root |
|---|---|---|---|---|---|---|
| word | independent | on | 98.29 | 95.10 | 89.12 | 84.36 |
| word | independent | off | 97.72 | 93.44 | 87.03 | 80.75 |
| word | chained | on | 98.29 | 96.60 | 89.72 | 86.46 |
| word | chained | off | 97.72 | 95.24 | 88.14 | 83.98 |
| word | oracle | on | 98.29 | 97.00 | 90.15 | 86.46 |
| word | oracle | off | 97.72 | 96.14 | 88.83 | 83.98 |
| root | independent | on | 98.28 | 95.11 | 89.10 | 84.31 |
| root | independent | off | 97.72 | 93.45 | 87.00 | 80.71 |
| root | chained | on | 98.28 | 96.59 | 89.65 | 86.38 |
| root | chained | off | 97.72 | 95.25 | 88.06 | 83.91 |
| root | oracle | on | 98.28 | 97.01 | 90.08 | 86.38 |
| root | oracle | off | 97.72 | 96.14 | 88.75 | 83.91 |

Table 4: Accuracy (%) of each radical and of the whole root, per configuration.

### 6.1 Analysis

The analysis below uses the split by word, the chained classifiers and the diacritic features, unless it says otherwise. The chained classifiers are the layered classifiers given the predicted positions of the earlier radicals. Table 5 breaks down the errors of that configuration.

**Layering.** Layering raises whole-root accuracy from 84.36% to 86.46%. The gain comes from the second and third radicals. The second radical rises from 95.10% to 96.60%. The third radical rises from 89.12% to 89.72%.

**Errors passed between classifiers.** The oracle variant gives the second and third classifiers the correct positions of the earlier radicals. It raises the second radical to 97.00% and the third radical to 90.15%. Its whole-root accuracy equals that of the chained classifiers, 86.46%. This equality holds by construction. A whole root is right only when the first radical is right. In that case the second classifier receives the correct position of the first radical under the chained variant and under the oracle variant. If the second radical is also right, the third classifier receives the correct positions of the first two radicals under each variant. So the errors passed from one classifier to the next cost accuracy on single radicals, not on whole roots.

**Unseen roots.** Splitting by root changes whole-root accuracy from 86.46% to 86.38%. The classifiers therefore depend very little on having seen a root before. The features describe the positions and the kinds of letters, not which letters they are.

**Diacritics.** Without the diacritic features, whole-root accuracy falls to 83.98%. The first radical falls from 98.29% to 97.72%. The second radical falls from 96.60% to 95.24%. The third radical falls from 89.72% to 88.14%.

**Radicals right per word.** All three radicals are right in 86.46% of the words. Exactly two are right in 12.12%. One radical is right in 0.99% of the words. No radical is right in 0.43%.

**Why the third radical is the hardest.** Our analysis finds two reasons. The first reason is that the third radical can occur in five positions, 3 to 7 (Table 2). The first and second radicals can each occur in four. Longer words offer more of these positions. Third-radical accuracy falls from 95.45% for three-letter words to 75.65% for seven-letter words. This fall is consistent with the first reason. The breakdown cannot separate it from other effects of word length. The second reason is that a weak third radical is hard to find. When the third radical is و, ي, ا or ى, its accuracy is 66.20%. For the other third radicals it is 92.80%. The data supports this reason directly.

**Radicals not in the word.** In 2,907 words, at least one radical is not in the word. The classifiers predict position 0 for every such radical in 63.61% of these words. The first radical is not in the word in 381 words. The first classifier detects 62.99% of them. For the second radical, the figures are 781 words and 44.17%. For the third radical, they are 1,894 words and 68.27%. Restoring these radicals is the step left for future work.

## 7 Application: separating lexemes in Lisan al-Arab

The extractor was first used in a parser for Lisan al-Arab, written in 2013. The parser turns the text of the dictionary into an XML database. Each entry becomes a `rootWord` element. Inside it, the parser separates lexemes from their meanings, elaborations and quotations.

| Group | Words | % |
|---|---|---|
| *Radicals right per word (share of all words)* | | |
| 3 of 3 | 22,439 | 86.46 |
| 2 of 3 | 3,145 | 12.12 |
| 1 of 3 | 257 | 0.99 |
| 0 of 3 | 111 | 0.43 |
| *Third radical accuracy, by weakness* | | |
| weak (و ي ا ى) | 3,003 | 66.20 |
| non-weak | 22,949 | 92.80 |
| *Third radical accuracy, by word length* | | |
| 2 letters | 32 | 100.00 |
| 3 letters | 7,391 | 95.45 |
| 4 letters | 3,508 | 92.59 |
| 5 letters | 10,450 | 87.61 |
| 6 letters | 3,540 | 85.17 |
| 7 letters | 1,031 | 75.65 |
| *Radical not in the word: detected as position 0* | | |
| radical 1 | 381 | 62.99 |
| radical 2 | 781 | 44.17 |
| radical 3 | 1,894 | 68.27 |
| all such radicals of a word | 2,907 | 63.61 |

Table 5: Error breakdown for the split by word, the layered (chained) classifiers and the diacritic features.

The hard part is deciding which words of an entry are derivatives of the entry's root. This is root verification. The root is given. The parser checks each word against it. A lexeme is the group of derivatives that one definition follows, such as a perfect verb, its imperfect and its verbal noun. A derivative marks the start of a lexeme. The text after the lexeme is its definition. The parser made this decision with two methods that voted.

The first method was the extractor described in this report. It found the root of a candidate word. The word was accepted when that root matched the entry's root.

The second method was a matching test. It normalized the root and the word first. It unfolded shadda, wrote آ as ءا, reduced every hamza to ء, and merged the three vowel letters into one. It then found the longest sequence of the root's letters that occurs in the word in the same order. The length of that sequence divided by the length of the root is the match ratio. A ratio of 1 is a full match.

The two methods had equal weight in the vote. When they disagreed, the total number of letters matched decided: more than three accepted the word, fewer than three rejected it. Four-lettered roots were judged by the matching test alone, because the extractor covers only three-lettered roots.

A hand check of a random sample of the parser's output gave the results in Table 6. The parser tags any sentence it cannot classify as an elaboration. The three missed meanings and the two missed lexemes were tagged that way. Two lemmas were assigned to the wrong root. The parser took ابتدأ as a derivative of بتأ. Its root is بدأ. It took منبت as a derivative of بتل. Its root is نبت.

The next planned use is word lookup in a Classical Arabic picture dictionary built from classical lexica such as Lisan al-Arab. A reader enters a word. The dictionary needs the word's root to find its entry.

| Element | Correct | Wrong | Missed |
|---|---|---|---|
| rootWord | 7 | 0 | 0 |
| individual lemmas | 87 | 2 | 10 |
| whole lexemes | 23 | 0 | 2 |
| meaning | 22 | 0 | 3 |
| elaboration | 15 | 5 | 0 |
| quotation | 8 | 0 | 0 |

Table 6: Hand check of a random sample of the Lisan al-Arab parser's output. These figures come from the 2013 report on the parser and were not re-run.

## 8 Future work

The features should be studied more deeply. They should be expounded and then filtered to perfectly fit this problem. There are a lot of attributes that this project kept out of its domain: attributes that are related to other types of derived words more complex than the presented 30 patterns. These types carry semantic information in their very morphology, which requires a higher degree of features.

For simplicity, the training set can be divided into:

- root vs. noun derivative;
- noun pattern vs. noun derivative;
- root vs. past verb;
- root and pattern vs. past verb;

and so on. A more challenging problem is to try all of that with completely undiacritized words.

The model should be able to extract roots not only from lemmas, but even from regular text.

The model must restore the radicals that are not in the word, which the classifiers already detect.

Moreover, the assumption that finding the three letters consists of three independent sub-problems needs to be revised; the layered classifiers are a first step.

Obviously, the model should cover derivatives of four-lettered roots in the future as well.

Another idea is to use both approaches simultaneously: the whole-root approach and the one-radical-per-classifier approach. Consider the following scenario. The whole-root classifier guessed an answer which conflicts with the first-radical classifier. Given that the latter has a 98% success rate, the former is most likely wrong. The two approaches can be joined by voting, weighted voting, or even an ANN (Wolpert, 1992).

## 9 Conclusion

We started the solution of the root extraction problem by designing an initial feature set consisting of attributes directly affecting the derivation mechanics. Then we discovered the idea of dividing the problem into three sub-problems, and so divided the feature set into three subsets. Most attributes are common between the three sets, but the real breakthrough is reducing the number of classes from thousands to only 24. Moreover, including the diacritic features raised the success rate of each radical, as Table 4 shows. Layering the classifiers raised whole-root accuracy further, to 86.46%.

## Acknowledgments

I thank Dr. Zubair Baig, in whose course the first version of this work was built. I also thank Dr. Lahouari Ghouti, in whose course the SVM version was built.

## References

Sabah S. Al-Fedaghi and Fawaz Al-Anzi. A new algorithm to generate Arabic root-pattern forms. In *Proceedings of the 11th National Computer Conference*, pages 391–400, Dhahran, 1989.

El Salam Al Hajjar, M. Hajjar, and K. Zreik. A system for evaluation of Arabic root extraction methods. In *Fifth International Conference on Internet and Web Applications and Services (ICIW)*, pages 506–512, 2010.

Hasan Al-Serhan and Aladdin Ayesh. A triliteral word roots extraction using neural network for Arabic. In *International Conference on Computer Engineering and Systems*, 2006a.

Hasan Al-Serhan and Aladdin Ayesh. An application of neural network for extracting Arabic word roots. In *Proceedings of the 10th WSEAS International Conference on Computers*, pages 637–641, Vouliagmeni, Athens, Greece, 2006b.

Hasan Muaidi Al-Serhan, R. Al Shalabi, and G. Kannan. New approach for extracting Arabic roots. In *Proceedings of the 2003 Arab Conference on Information Technology (ACIT)*, pages 42–59, 2003.

Riyad Al-Shalabi. Pattern-based stemmer for finding Arabic roots. *Information Technology Journal*, 4(1):38–43, 2005.

Riyad Al-Shalabi and Martha Evens. A computational morphology system for Arabic. In *Proceedings of the Workshop on Computational Approaches to Semitic Languages (Semitic '98)*, pages 66–72, 1998.

Emad Al-Shawakfa et al. A comparison study of some Arabic root finding algorithms. *Journal of the American Society for Information Science and Technology*, 61(5):1015–1024, 2010.

Imad A. Al-Sughaiyer and Ibrahim A. Al-Kharashi. Arabic morphological analysis techniques: A comprehensive survey. *Journal of the American Society for Information Science and Technology*, 55(3):189–213, 2004.

ALECSO. Nizam al-ishtiqaq wa al-tasrif fi al-lugha al-arabiyya (derivation and conjugation system for Arabic), version 1. Arab League Educational, Cultural and Scientific Organization, 2007.

Alkhalil. Alkhalil morphological analyzer. `http://www.alecso.org.tn/index.php?option=com_content&task=view&id=1302&Itemid=998&lang=ar`, 2010.

Mohamed Altantawy, Nizar Habash, and Owen Rambow. Fast yet rich morphological analysis. In *Proceedings of the 9th International Workshop on Finite State Methods and Natural Language Processing*, pages 116–124, Blois, France, 2011.

Mohammed Attia, Pavel Pecina, Antonio Toral, Lamia Tounsi, and Josef van Genabith. A lexical database for Modern Standard Arabic interoperable with a finite state morphological transducer. In *Systems and Frameworks for Computational Morphology*, volume 100 of *Communications in Computer and Information Science*, pages 98–118. Springer, 2011.

Abderrezak Brahmi, Ahmed Ech-Cherif, and Abdelkader Benyettou. An Arabic lemma-based stemmer for latent topic modeling. *The International Arab Journal of Information Technology*, 2011.

Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3):27:1–27:27, 2011.

Ezra Daya, Dan Roth, and Shuly Wintner. Learning Hebrew roots: Machine learning with linguistic constraints. In *Proceedings of EMNLP*, pages 357–364, Barcelona, Spain, 2004.

Ezra Daya, Dan Roth, and Shuly Wintner. Identifying Semitic roots: Machine learning with linguistic constraints. *Computational Linguistics*, 34(3):429–448, 2008.

Tarek El-Shishtawy and Fatma El-Ghannam. An accurate Arabic root-based lemmatizer for information retrieval purposes. *International Journal of Computer Science Issues*, 9(1), 2012.

Ahmed Khorsi. Effective unsupervised Arabic word stemming: Towards an unsupervised radicals extraction. *The International Arab Journal of Information Technology*, 2010.

David H. Wolpert. Stacked generalization. *Neural Networks*, 5:241–260, 1992.